

COLECCIÓN DE APLICACIONES GRATUITAS PARA CONTEXTOS EDUCATIVOS

Robótica Educativa

Kit Básico Arduino UNO



Buenos Aires Ciudad



Vamos Buenos Aires

Prólogo

Este tutorial se enmarca dentro de los lineamientos del Plan Integral de Educación Digital (PIED) del Ministerio de Educación del Gobierno de la Ciudad Autónoma de Buenos Aires que busca integrar los procesos de enseñanza y de aprendizaje de las instituciones educativas a la cultura digital.

Uno de los objetivos del PIED es “fomentar el conocimiento y la apropiación crítica de las Tecnologías de la Información y de la Comunicación (TIC) en la comunidad educativa y en la sociedad en general”.

Cada una de las aplicaciones que forman parte de este banco de recursos son herramientas que, utilizándolas de forma creativa, permiten aprender y jugar en entornos digitales. El juego es una poderosa fuente de motivación para los alumnos y favorece la construcción del saber. Todas las aplicaciones son de uso libre y pueden descargarse gratuitamente de Internet e instalarse en cualquier computadora.

De esta manera, se promueve la igualdad de oportunidades y posibilidades para que todos puedan acceder a herramientas que desarrollen la creatividad.

En cada uno de los tutoriales se presentan “consideraciones pedagógicas” que funcionan como disparadores pero que no deben limitar a los usuarios a explorar y desarrollar sus propios usos educativos.

La aplicación de este tutorial no constituye por sí misma una propuesta pedagógica. Su funcionalidad cobra sentido cuando se integra a una actividad. Cada docente o persona que quiera utilizar estos recursos podrá construir su propio recorrido.

Índice

- [¿Qué es?](#)
- [Requerimientos técnicos](#)
- **Nociones básicas**
 - [Entradas y salidas digitales o analógicas](#)
 - [Construcción y control](#)
 - [Elementos del kit](#)
 - [Componentes de conexión](#)
 - [Componentes de entrada de datos](#)
 - [Componentes de salida de datos](#)
 - [Materiales y componentes accesorios](#)
- **Actividades**
 - [Actividad 1. Encendido de LED RGB con pulsadores](#)
 - [Componentes del kit](#)
 - [Construcción](#)
 - [Programación en mBlock](#)
 - [Programación en Arduino IDE](#)
 - [Programación en Ardublock](#)
 - [Actividad 2. Flecha selectora](#)
 - [Componentes del kit](#)
 - [Construcción](#)
 - [Programación en mBlock](#)
 - [Programación en Arduino IDE](#)
 - [Programación en Ardublock](#)
 - [Actividad 3. Encendido de leds cuando se oscurece el ambiente](#)
 - [Componentes del kit](#)
 - [Construcción](#)
 - [Programación en mBlock](#)
 - [Programación en Arduino IDE](#)
 - [Programación en Ardublock](#)
- [Enlaces de interés](#)

¿Qué es?

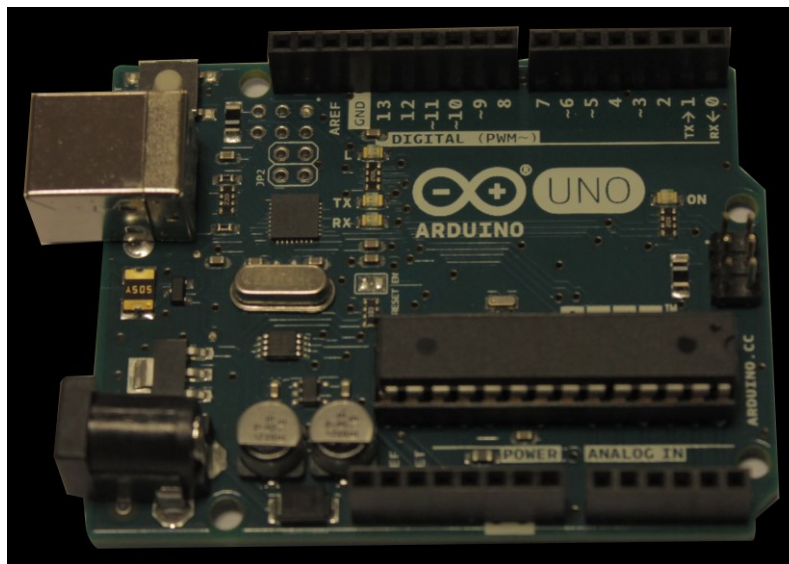
Arduino es una plataforma de electrónica abierta para la creación de prototipos basada en software y hardware flexibles y fáciles de usar. Fue creado para aficionados e interesados en crear entornos u objetos interactivos.

Las placas Arduino pueden leer entradas generadas de diversas formas: luz en un sensor, un botón presionado o un mensaje de Twitter, entre otros. Y convertirlo en una salida, por ejemplo: activar un motor, encender un LED o publicar algo en línea.

Utiliza el entorno de desarrollo Arduino basado en el lenguaje de programación Processing.

Arduino cuenta con diferentes tipos de placas. El kit que se analizará en este tutorial cuenta con la placa Arduino UNO.

Para conocer los principios básicos del uso de Arduino se sugiere la lectura del siguiente [Tutorial](#) disponible en el Campus Virtual de Educación Digital.



Requerimientos técnicos

- Puede utilizarse bajo los sistemas operativos Windows y GNU/Linux.
- Kit Arduino UNO.
- Cable USB para conectar la placa a la computadora.
- Software Arduino. URL de la aplicación: <http://www.arduino.cc>



Nociones básicas

Entradas y salidas digitales o analógicas

A la placa Arduino se conectan elementos que ingresan datos (entradas) o bien que envían datos (salidas).

Entradas: pueden ser analógicas o digitales. Tienen como propósito leer los sensores que se conectan a la placa Arduino.

Las **entradas analógicas** convierten una magnitud con una amplia variedad de valores en un dato digital (0 o 1) puesto que Arduino trabaja sólo con valores digitales. Esta conversión se realiza con un conversor A/D interno de la placa. Así es como estas entradas pueden leer tensión o intensidad. Ingresan a la placa Arduino UNO a través de los pines que van del A0 al A5.

Las **entradas digitales** trabajan con señales de tensión, cuando ingresan por dicha entrada 5 voltios se interpreta como un "1" y cuando ingresa cero voltios se interpreta como un "0". Ingresan a la placa Arduino UNO a través de los pines que van del 0 al 13.

Salidas: pueden ser analógicas o digitales. Tienen como propósito enviar una señal a los actuadores que se conectan a la placa Arduino para su ejecución.

Las **salidas digitales**, al igual que las entradas, trabajan con señales de tensión, cuando envían 5 voltios se interpreta como un "1" y cuando ingresa cero voltios se interpreta como

un "0". Estos datos salen de la placa Arduino UNO a través de los pines que van del 0 al 13, al igual que las entradas digitales. Por esta razón es necesario a través de la programación especificar si el pin será usado de entrada o de salida.

Las **salidas analógicas** se realizan a través de los pines PWM (Pulse Width Modulation – modulación por ancho de pulso). La misma permite obtener valores intermedios de salidas entre 0 y 5V escalados en 255 niveles, siendo 0 el más bajo (0 V) y 255 el más alto (5 V). Esto funciona como un conversor D/A. En la placa Arduino UNO los pines que permiten esta función están señalados con el símbolo ~, en este caso son los pines 3, 5, 6, 9, 10 y 11.



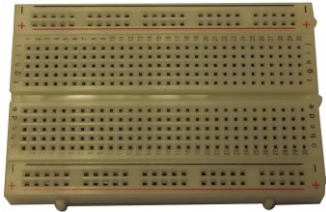
Construcción y control

Cada proyecto donde se involucra la placa Arduino requiere de dos momentos particulares: **construcción del proyecto y automatización y control del trabajo construido** a través de la programación. Esto está desarrollado en las distintas actividades explicadas en el presente tutorial.



Elementos del kit

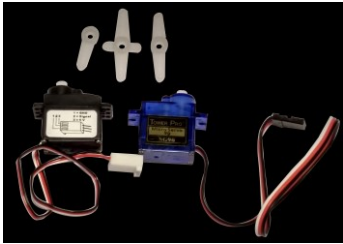
El kit que se explica en el siguiente documento es un Kit Básico, no implica que todos los kits posean los mismos elementos. El único elemento en común es la placa **Arduino UNO**.

Componentes de conexión	
Placa de pruebas <i>Protoboard o Breadboard.</i>  Consiste en una placa con un interior metálico, sobre la cual se pueden montar componentes electrónicos. Cada orificio está interconectado con otros, con una distribución específica de conexión. No todos están conectados entre sí.	Cables puente  Se utilizan para conectar componentes entre sí sobre la placa de prueba y a la placa de Arduino.
Cable USB  Permite conectar la placa Arduino Uno a una computadora para que se pueda programar. También proporciona la alimentación necesaria tanto a la placa Arduino como a los componentes electrónicos que forman parte de los proyectos de este kit.	Clip para alimentación  Permite conectar una batería de 9V o un porta pilas. Se utiliza cuando el proyecto construido necesita más energía que la brindada por la placa Arduino conectada al USB, o cuando se le quiere dar autonomía al proyecto sin estar conectado a la computadora.

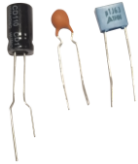
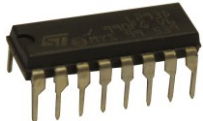
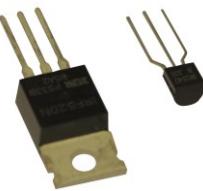


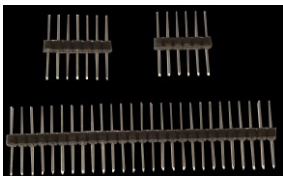
Componentes de entrada de datos		
Preset o Potenciómetro  Es una resistencia variable. Consiste en una pista resistiva con conexiones a ambos extremos y un cursor o punto medio que se mueve a lo largo de la pista según gira el eje, modificando el valor de entrada. En los <i>preset</i> el punto medio se mueve con un destornillador o pestaña.	Sensor de luz o fotoresistor  También llamada fotocélula o resistencia dependiente de la luz. Se trata de una resistencia variable que cambia su resistencia según el nivel de luz que incide sobre su superficie.	Pulsador  Interrupor momentáneo que cierran un circuito cuando es presionado y desconecta el mismo cuando es liberado.
Sensor de inclinación  Interrupor que se abre o se cierra dependiendo de su orientación. Es un cilindro hueco con una bola de metal en su interior que hará que los dos terminales se unan a través de esta bola cuando se incline.	Sensor de temperatura  Cambia el valor de salida dependiendo de la temperatura, es decir según este componente se encuentre más caliente o más frío.	Fototransistor  Genera una corriente proporcional a la cantidad de luz que recibe. Disponible en el kit original de Arduino versión 2017.



Componentes de salida de datos		
LED (Diodo emisor de luz)  Tipo de diodo que emite luz cuando la corriente lo atraviesa. Como en todos los diodos, la corriente solo fluye en un sentido a través de estos componentes. El terminal (pata) positivo de la alimentación, es generalmente el más largo, mientras que el negativo es el más corto.	Pantalla de cristal líquido (Display LCD)  Pantalla numérica o gráfica basada en cristal líquido. Los LCDs están disponibles en varios tamaños, formas y estilos. El que se incluye con este kit dispone de 2 filas con 16 caracteres en cada una de ellas.	Servo motor  Motor cuyo eje sólo puede girar 180 grados. A través de dispositivos de entrada o programación se le indica al motor a qué posición angular se debe de mover. Trae diferentes brazos accesorios para ajustar al eje según el proyecto a realizar.
Motor continuo  Convierte la energía eléctrica en energía mecánica cuando la electricidad es aplicada a sus dos terminales. Su eje tiene un giro continuo.	LED RGB  Led que enciende en tres colores diferentes: rojo (R de Red), verde (G de green), azul (B de blue). Tiene 4 terminales, uno para cada color y un cuarto terminal que puede ser el positivo común o el negativo común.	Buzzer  Componente electrónico que genera sonidos.



Materiales y componentes accesorios		
Capacitores  Estos componentes almacenan y devuelven energía eléctrica en un circuito. Cuando el voltaje del circuito es más alto que el que está almacenado en el condensador, la corriente fluye del circuito al condensador, dándole una carga. Cuando la tensión del circuito es más baja, la energía eléctrica almacenada en el condensador es devuelta al circuito. A menudo se colocan entre los terminales positivo y negativo de una alimentación de un sensor o un motor para ayudar a suavizar las fluctuaciones de tensión que se puedan producir.	Diodo  Conduce la electricidad en una sola dirección. Es útil usarlo en un circuito con un motor continuo. Tiene polaridad, generalmente el negativo y se distingue por una línea de color blanco o gris.	Puente H  Se trata de un circuito que permite controlar la polaridad de la tensión. Generalmente se utiliza para controlar la dirección de giro de un motor.
Ópticoacoplador  Reemplaza a un interruptor, permitiendo conectar dos circuitos. En su interior hay un pequeño led que, cuando se ilumina, hace que un fotoreceptor cierre un interruptor interno.	Resistencia fija  Se opone al paso de la corriente eléctrica en un circuito dando como resultado un cambio en la tensión y en la corriente. El valor de las resistencias se mide en ohm. La combinación de las bandas de colores indica su valor.	Transistores  Puede operar como un interruptor electrónico. Es útil para controlar componentes de alta corriente/tensión como los motores y para amplificar la corriente.

Libro de proyectos  Disponible solamente en los kits originales de Arduino. Libro con proyectos y descripción de elementos.	Tira de pines macho  Ayudan a realizar las conexiones de forma más fácil. Se pueden separar para soldar o conectar de forma individual.	Material complementario  Material troquelable para la realización de los proyectos detallados en el libro de proyectos. Disponible en los kits originales.
---	---	--



Actividades

Actividad 1. Encendido de LED RGB con pulsadores

El proyecto consiste en un LED emisor de luz RGB que enciende en color rojo, verde o azul según el pulsador presionado.

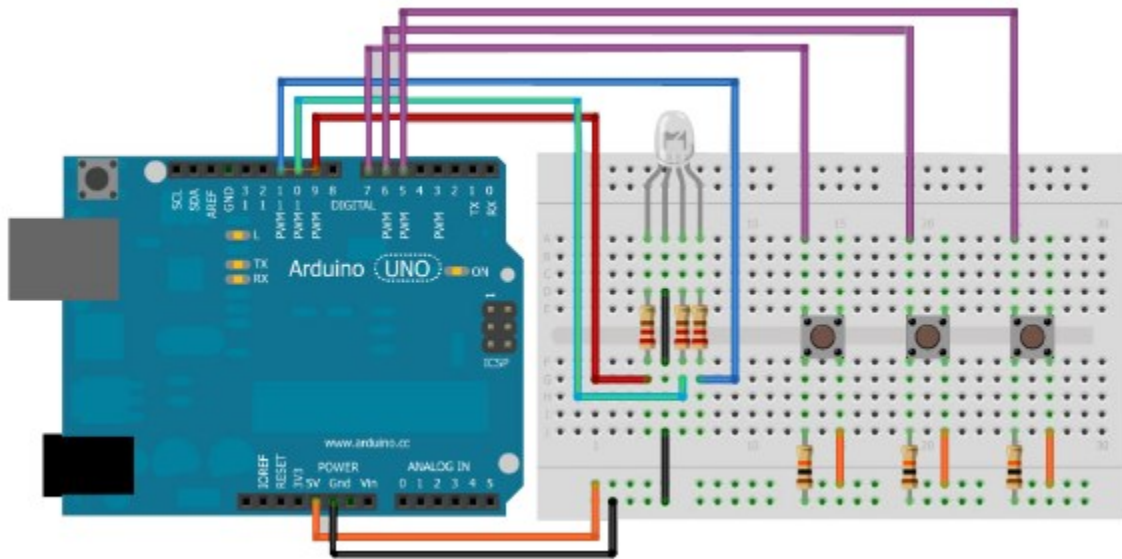
Componentes del kit que se utilizan

Bloques de entradas de datos	3 pulsadores
Bloques de salida de datos	Led RGB
Conectores	Cable USB Cables puente Protoboard
Componentes accesorios	3 resistencias fijas de 220 Ohms 3 resistencias fijas de 10K Ohms



Construcción

Conectar los componentes como muestra la imagen.



LED RGB:

- Cátodo común (conector más largo del led) a GND de la placa.
- Conector corto 1 a una resistencia de 220Ω y de allí al pin 9 de la placa.
- Conector corto 2 a una resistencia de 220Ω y de allí al pin 10 de la placa.
- Conector corto 3 a una resistencia de 220Ω y de allí al pin 11 de la placa.

Botones pulsadores:

Una pata conectora de cada pulsador a una resistencia de $10K\Omega$ a GND de la placa. Misma pata conectada al pin 5, 6 y 7 de la placa, un pin conectado a cada uno de los pulsadores. La otra pata conectada al positivo (5V) de la placa.



Programación en mBlock



Programación en Arduino IDE

```
/*
Led RGB
Controla cada color del led RGB con un pulsador.
*/

//Variables estáticas
int ledRojo = 9; //Declaro pin para cátodo del rojo
int ledVerde = 10; //Declaro pin para cátodo del verde
int ledAzul = 11; //Declaro pin para cátodo del azul
int pulsador1 = 5; //Declaro pin para el pulsador1
int pulsador2 = 6; //Declaro pin para el pulsador2
int pulsador3 = 7; //Declaro pin para el pulsador3
//Variables dinámicas
int estado1; //Inicializo la variable de estado1
int estado2; //Inicializo la variable de estado2
int estado3; //Inicializo la variable de estado3

void setup() {
  pinMode(ledRojo, OUTPUT); //Configuro el pin 9 como salida
  pinMode(ledVerde, OUTPUT); //Configuro el pin 10 como salida
  pinMode(ledAzul, OUTPUT); //Configuro el pin 11 como salida
  pinMode(pulsador1, INPUT); //Configuro el pin 5 como entrada
  pinMode(pulsador2, INPUT); //Configuro el pin 6 como entrada
  pinMode(pulsador3, INPUT); //Configuro el pin 7 como entrada
}

void loop() {
  estado1 = digitalRead(pulsador1); //Lee el estado del pin5 y lo almacena
  estado2 = digitalRead(pulsador2); //Lee el estado del pin6 y lo almacena
  estado3 = digitalRead(pulsador3); //Lee el estado del pin7 y lo almacena

  if(estado1 == HIGH){          //Si el pulsador1 está pulsado
    digitalWrite(ledRojo, HIGH); //Enciende led rojo
  }
  else{                          //Si no está pulsado
```

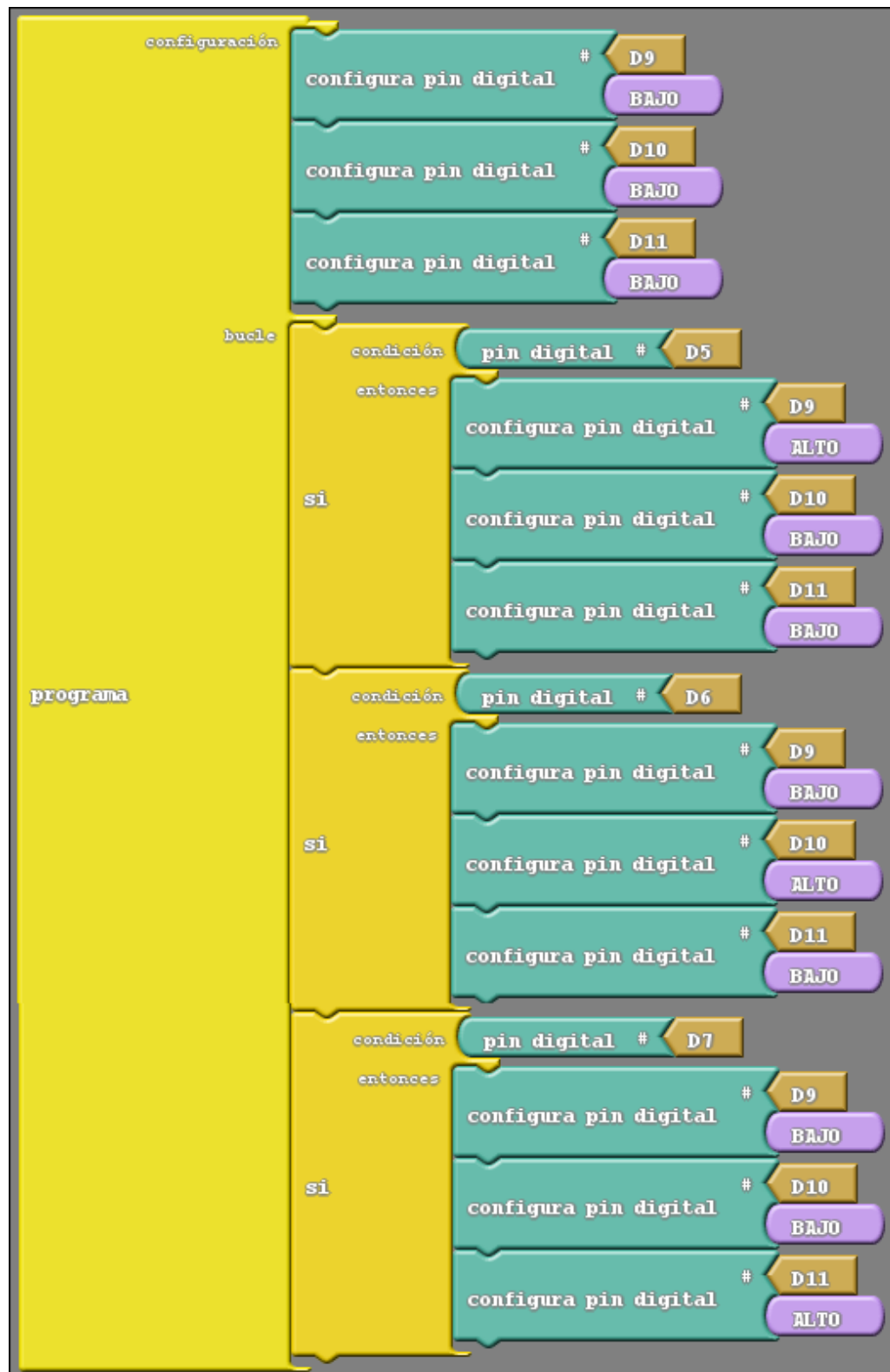
```
digitalWrite(ledRojo, LOW);    //Apaga led rojo
}

if(estado2 == HIGH){           //Si el pulsador2 está pulsado
  digitalWrite(ledVerde, HIGH); //Enciende led verde
}
else{                           //Si no está pulsado
  digitalWrite(ledVerde, LOW);   //Apaga led verde
}

if(estado3 == HIGH){           //Si el pulsador3 está pulsado
  digitalWrite(ledAzul, HIGH);  //Enciende led azul
}
else{                           //Si no está pulsado
  digitalWrite(ledAzul, LOW);    //Apaga led azul
}
}
```



Programación en Ardublock



Actividad 2. Flecha selectora

Servomotor que tiene adherida una flecha selectora sobre un tablero con temáticas diferentes.

Al presionar un pulsador la flecha selecciona de forma aleatoria un tema.

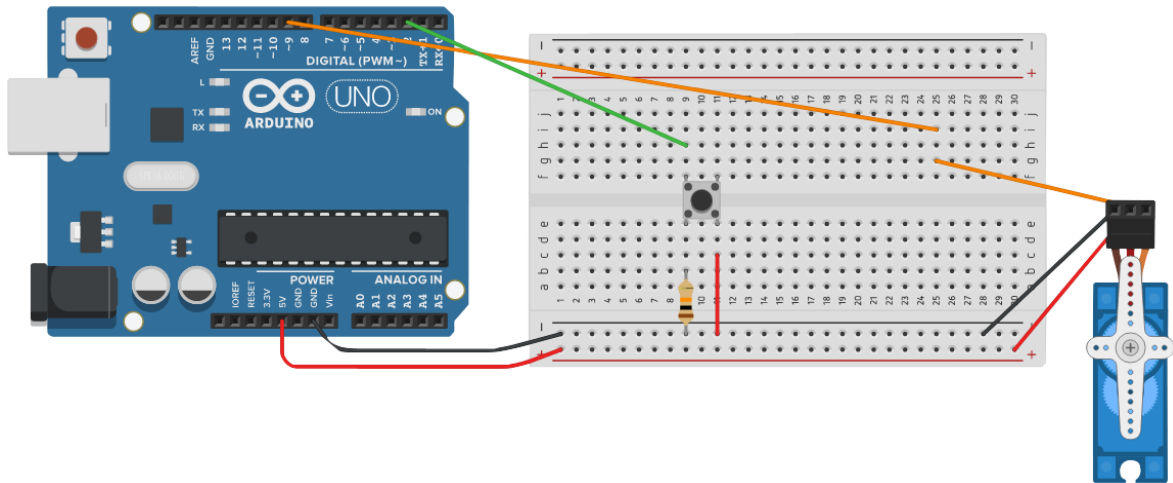
Componentes del kit que se utilizan

Bloques de entradas de datos	Pulsador
Bloques de salida de datos	Servomotor
Conectores	Cable USB Cables puente Protoboard
Componentes accesorios	1 resistencia fija de 10KOhms



Construcción

Conectar los componentes como indica la imagen.



Servomotor:

- Cable marrón al GND (negativo) de la placa
- Cable rojo a los 5V (positivo) de la placa
- Cable naranja al pin digital 9 de la placa

Botón pulsador:

Una pata conectada a una resistencia de 10KΩ a GND de la placa. Misma pata conectada al pin 2 de la placa. La otra pata conectada al positivo (5V) de la placa.



Programación en mBlock



Programación en Arduino IDE

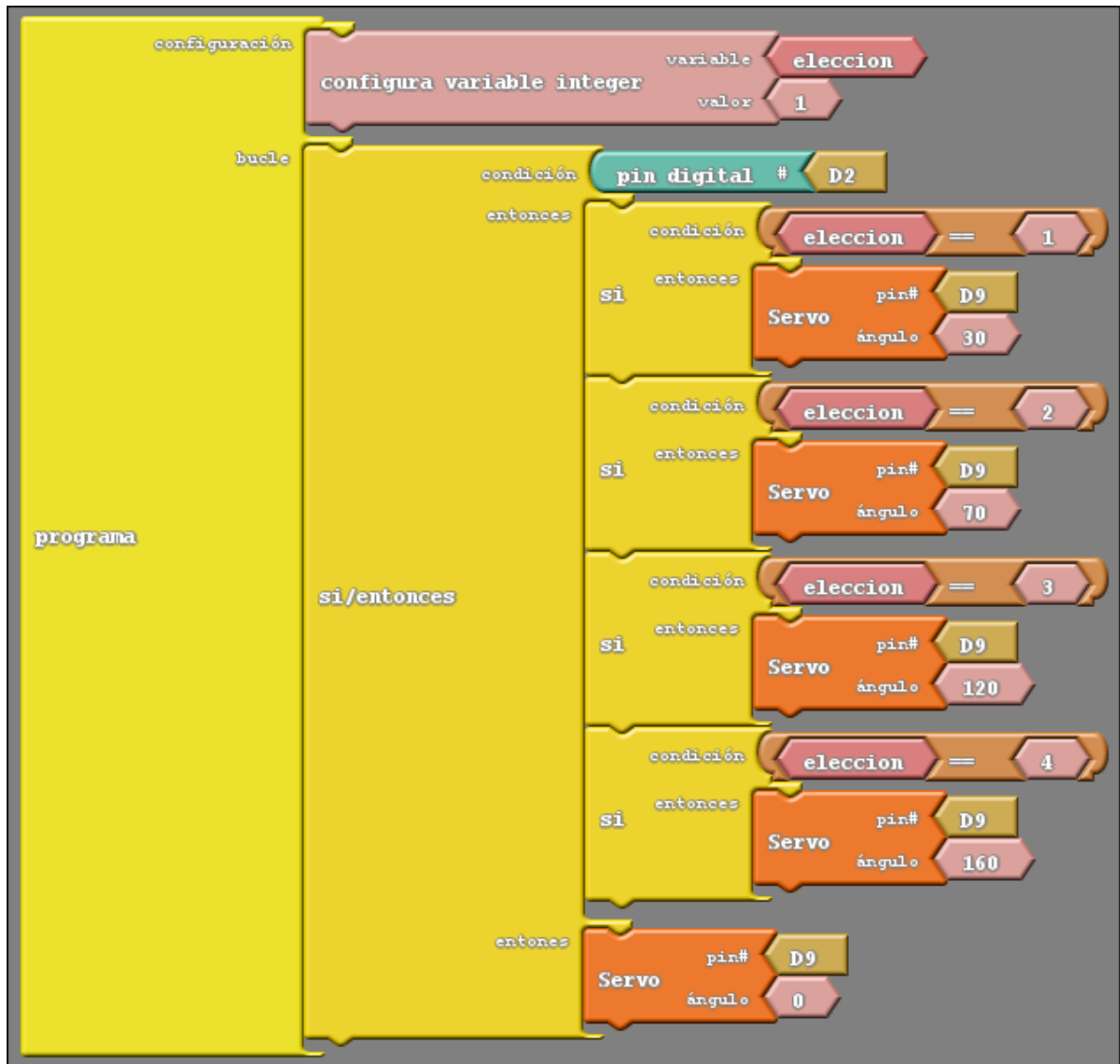
```
#include <Servo.h> // librería necesaria para el funcionamiento del servo
Servo myservo; //Habilita el servo
int boton = 2; // variable donde se conecta el botón
int eleccion; //variable donde se almacena el número random

void setup() {
  myservo.attach(9); // Conecta el servo al pin 9
}

void loop() {
  boton = digitalRead (2); //lee el botón
  if (boton){           //Si el botón está presionado
    myservo.write(0);   //lleva el servomotor a 0°
    eleccion = random(1,4); //elige un número al azar entre 1 y 4
    if (eleccion == 1) {
      myservo.write(30); // Si es 1 lleva el servomotor a 30°
      delay(15);
    }
    if (eleccion == 2) {
      myservo.write(70); // Si es 2 lleva el servomotor a 70°
      delay(15);
    }
    if (eleccion == 3) {
      myservo.write(120); // Si es 3 lleva el servomotor a 120°
      delay(15);
    }
    if (eleccion == 4) {
      myservo.write(170); // Si es 4 lleva el servomotor a 170°
      delay(15);
    }
  }
}
```



Programación en Ardublock



Actividad 3. Encendido de leds cuando se oscurece el ambiente

Secuencia de leds que se encienden progresivamente según la intensidad de luz recibida. Cuanto menor sea la luz, mayor cantidad de leds se encienden.

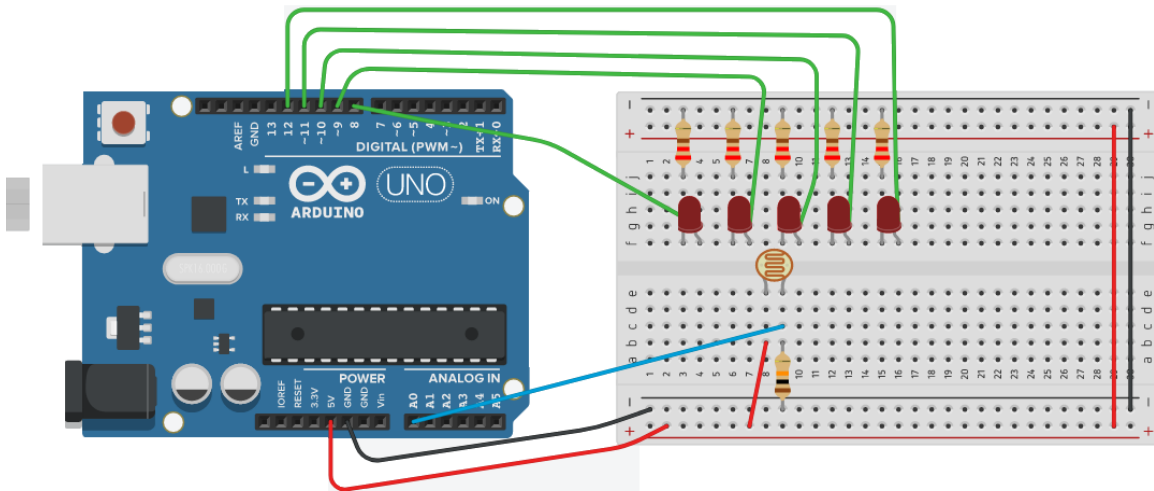
Componentes del kit que se utilizan

Bloques de entradas de datos	Sensor de luz
Bloques de salida de datos	5 leds
Conectores	Cable USB Cables puente Protoboard
Componentes accesorios	5 resistencias fijas de 220 Ohms 1 resistencia fija de 10KOhms



Construcción

Conectar los componentes como indica la imagen.



Leds:

Las patas positivas (la más larga) de cada led conectada a los pines digitales 8, 9, 10, 11 y 12 respectivamente de la placa.

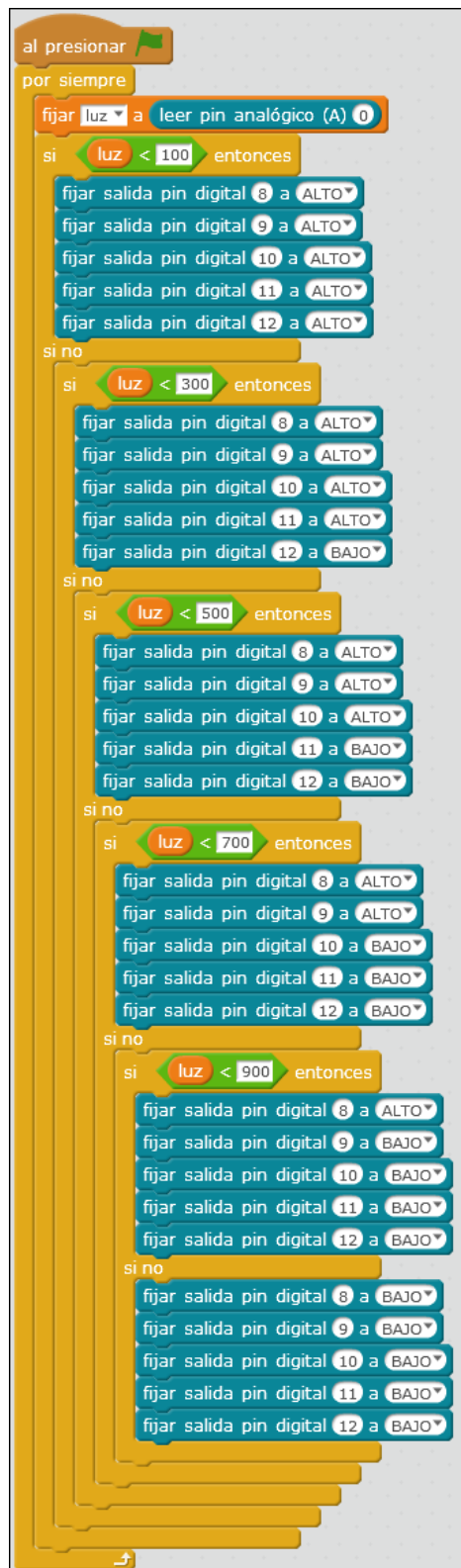
Las patas negativas van conectadas cada una a una resistencia de 220Ω y de allí al GND (negativo) de la placa.

LDR Resistencia lumínica:

- Una pata conectada al positivo (5V) de la placa.
- La otra pata conectada a una resistencia de $10\text{ K}\Omega$.



Programación en mBlock



Programación en Arduino IDE

```
int sensorPin = A0;
int ledPin1 = 8;
int ledPin2 = 9;
int ledPin3 = 10;
int ledPin4 = 11;
int ledPin5 = 12;
int luz = 0;

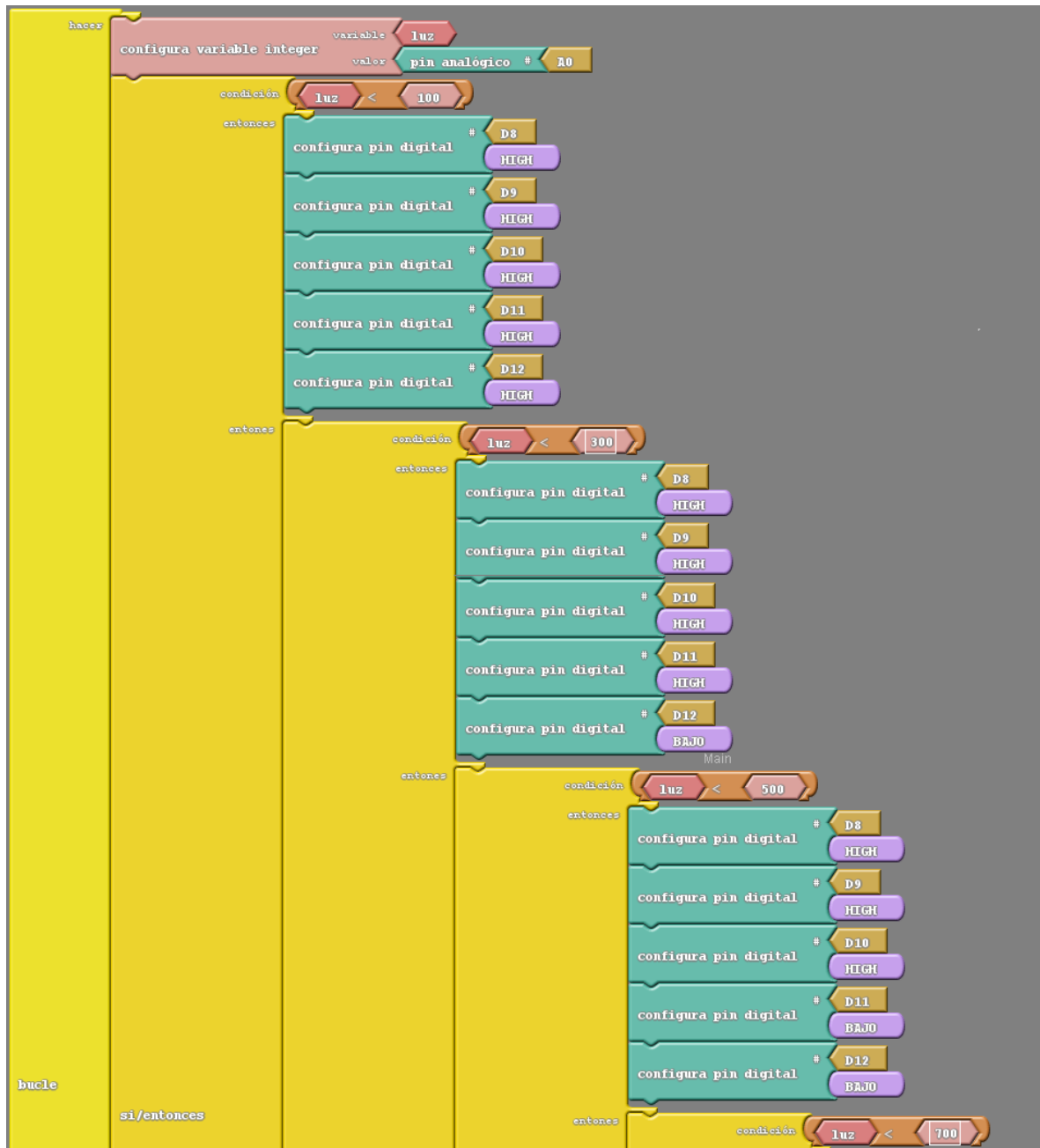
void setup() {
  // declare the ledPin as an OUTPUT:
  pinMode(ledPin1, OUTPUT);
  pinMode(ledPin2, OUTPUT);
  pinMode(ledPin3, OUTPUT);
  pinMode(ledPin4, OUTPUT);
  pinMode(ledPin5, OUTPUT);
}

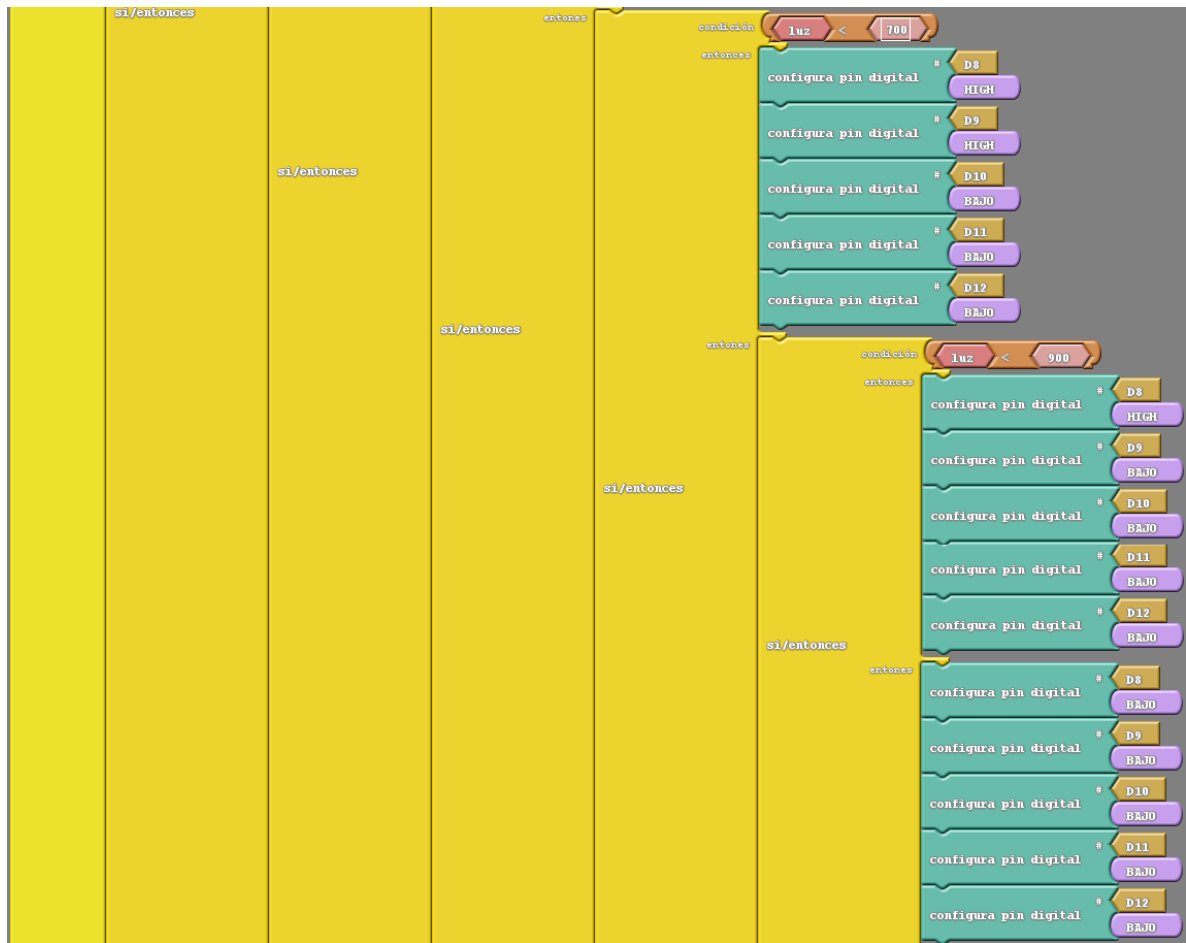
void loop() {
  // read the value from the sensor:
  luz = analogRead(sensorPin);
  if (luz<100){
    digitalWrite(ledPin1, HIGH);
    digitalWrite(ledPin2, HIGH);
    digitalWrite(ledPin3, HIGH);
    digitalWrite(ledPin4, HIGH);
    digitalWrite(ledPin5, HIGH);
  } else {
    if (luz<300){
      digitalWrite(ledPin1, HIGH);
      digitalWrite(ledPin2, HIGH);
      digitalWrite(ledPin3, HIGH);
      digitalWrite(ledPin4, HIGH);
      digitalWrite(ledPin5, LOW);
    } else {
      if (luz<500){
```

```
digitalWrite(ledPin1, HIGH);  
digitalWrite(ledPin2, HIGH);  
digitalWrite(ledPin3, HIGH);  
digitalWrite(ledPin4, LOW);  
digitalWrite(ledPin5, LOW);  
} else {  
  if (luz<700){  
    digitalWrite(ledPin1, HIGH);  
    digitalWrite(ledPin2, HIGH);  
    digitalWrite(ledPin3, LOW);  
    digitalWrite(ledPin4, LOW);  
    digitalWrite(ledPin5, LOW);  
  } else {  
    if (luz<900){  
      digitalWrite(ledPin1, HIGH);  
      digitalWrite(ledPin2, LOW);  
      digitalWrite(ledPin3, LOW);  
      digitalWrite(ledPin4, LOW);  
      digitalWrite(ledPin5, LOW);  
    } else {  
      digitalWrite(ledPin1, LOW);  
      digitalWrite(ledPin2, LOW);  
      digitalWrite(ledPin3, LOW);  
      digitalWrite(ledPin4, LOW);  
      digitalWrite(ledPin5, LOW);  
    }  
  }  
}  
}  
}}
```



Programación en Ardublock





Enlaces de interés

- Sitio oficial Arduino: <http://www.arduino.cc>
- Material de consulta: <https://campusvirtualintec.buenosaires.gob.ar/course/view.php?id=383>



Contacto:

ayuda.pedagogico.digital@bue.edu.ar



Esta obra se encuentra bajo una Licencia Attribution-NonCommercial-ShareAlike 2.5 Argentina de Creative Commons. Para más información visite <http://creativecommons.org/licenses/by-nc-sa/2.5/ar/>



Vamos Buenos Aires